

IIB43203 ASSESSMENT COVERSHEET

Attach this coversheet as the cover of your submission. All sections must be completed.

Section A: Submission Details

Programme : Bachelor In Internet Of Things
Course Code & Name : IIB43203 - CLOUD COMPUTING
Assessment : **GROUP PROJECT**
Dateline : Day 07 Month 06 Year 2026
Penalties :

- **10%** will be deducted for late submission.
- Plagiarised work is an Academic Offence in University Rules & Regulations and will be penalised accordingly.
- **AI content** will affect your mark

Section B: Academic Integrity

Tick (✓) each box below if you agree:

☒	I have read and understood the UniKL's policy on Plagiarism in University Rules & Regulations.
☒	This submission is my own, unless indicated with proper referencing.
☒	This submission has not been previously submitted or published.
☒	This submission follows the requirements stated in the course.

Section C: Submission Detail

Office Receipt of Submission

No	Student Name(s)	Student ID(s)
1	AMEER AFIQ BIN ARJUNA	52227124345
2	AHMAD FARHAN BIN RIZMAL	52227124355
3	MUHAMMAD FAZIL BIN FIHIR	52227124253
4	ISMAIRIL BIN MUHAMMAD RIZAL	52227124581

Student Receipt of Submission

The submission receipt of the student task will be based on the **VLE submission date and time**.



MALAYSIAN INSTITUTE OF INFORMATION TECHNOLOGY

MARCH 2026 SEMESTER

CLOUD COMPUTING (IIB43203)

GROUP PROJECT

PREPARED BY:

NO	NAME	STUDENT ID NO.	HP NO.
1	AMEER AFIQ BIN ARJUNA	52227124345	0135121043
2	AHMAD FARHAN BIN RIZMAL	52227124355	0183557561
3	MUHAMMAD FAZIL BIN FIHIR	52227124253	0124338804
4	ISMAIRIL BIN MUHAMMAD RIZAL	52227124581	0148841822

EVALUATE BY:

Dr. Megat Norulazmi

SUBMISSION DATE:

07/06/2026

TABLE OF CONTENT

NO.	Content	Page
i	Table of Content	1
1	Introduction	2
2	Project Objectives & Scope	2
3	Team Roles & Responsibilities	2
4	Cloud Architecture Design	2
4.1	Networking (Amazon VPC)	3
4.2	Compute (Amazon EC2)	3
4.3	Database (Amazon RDS)	3
4.3	Storage (Amazon S3)	3
5	Security & Cost Optimization	3-4
6	Step-by-Step Implementation Guide	4-20
6.1	Step 1: VPC & Subnets	4-6
6.2	Step 2: Security Groups	6-8
6.3	Step 3: Database Instance Creation	8-11
6.4	Step 4: Creating Your Web Server with EC2	11-13
6.5	Step 5: Test Your Web Server	13-17
6.6	Step 6: Create Amazon S3 Bucket	17-18
6.7	Step 7: Create IAM roles for EC2 access to Amazon S3	18-20
7	Testing & Validation	20-21
8	Challenges & Resolutions	21
9	Project Deliverables	21-22

1.0 - Introduction

This project demonstrates the successful design, deployment, and configuration of a scalable web application utilizing Amazon Web Services (AWS). Leveraging the AWS Free Tier, our team engineered a standard three-tier architecture that distinctly separates the presentation, application, and data layers. By utilizing Amazon EC2, RDS, VPC, and S3, we established a secure, highly available environment capable of serving dynamic web content while protecting sensitive database information within a private network.

2.0 - Project Objectives & Scope

The primary objective of this project is to gain hands on experience with core cloud computing concepts. The specific goals include:

- Infrastructure as a Service (IaaS) is to deploy virtual servers to host web application logic.
- Platform as a Service (PaaS) by utilizing managed relational databases to reduce administrative overhead.
- Network Isolation is for designing a secure Virtual Private Cloud (VPC) with public and private subnets.
- Decoupled Storage for offloading static web assets to scalable object storage to optimize compute performance.
- Collaborative Development by utilizing GitHub for version control and teamwork.

3.0 - Team Roles & Responsibilities

To ensure equal participation and meet the marking criteria for teamwork, tasks were divided into specialized roles:

Team Member	Designated Role	Core Responsibilities
AMEER AFIQ BIN ARJUNA	Cloud Network Architect	Designed the VPC, configured subnets, Internet Gateways, Route Tables, and overall CIDR block planning.
AHMAD FARHAN BIN RIZMAL	Database Administrator	Provisioned Amazon RDS (MySQL), created database subnet groups, and established database credentials.
MUHAMMAD FAZIL BIN FIHIR	Systems Administrator	Deployed EC2 instances, wrote the bash automation scripts (User Data) for PHP, and configured EC2 Security Groups.
ISMAIRIL BIN MUHAMMAD RIZAL	Storage & Media Lead	Managed Amazon S3 buckets, configured IAM/Bucket policies for public access, and finalized the video presentation editing.

4.0 - Cloud Architecture Design

4.1 - Networking (Amazon VPC)

- The foundational network utilizes a custom IPv4 CIDR Block of 10.0.0.0/16.
- Public Subnets: Deployed across two separate Availability Zones (10.0.1.0/24 and 10.0.2.0/24). These subnets are attached to an Internet Gateway via Route Tables, providing public internet access for our front-facing web servers.
- Private Subnets: Deployed across two Availability Zones (10.0.3.0/24 and 10.0.4.0/24) to strictly secure our backend data layer. These subnets have no direct inbound internet access. Outbound internet access for software updates is routed through a NAT Gateway deployed in a single Availability Zone.

4.2 - Compute (Amazon EC2)

- The application layer is powered by Amazon Elastic Compute Cloud. We provisioned a t3.micro instance running the Amazon Linux 2023 operating system.
- The instance is located in a Public Subnet and is auto-assigned a Public IPv4 address to serve client requests.
- An automated bash script (User Data) was utilized during launch to dynamically install the Apache web server (httpd), PHP, and MySQL connectivity tools, as well as to clone the application source code directly from a GitHub repository.

4.3 - Database (Amazon RDS)

- The data layer utilizes a managed relational database system running the MySQL engine on a db.t3.micro instance class.
- For maximum security, the database is placed entirely within the Private Subnets using a dedicated DB Subnet Group.
- The database has no public IP address and is strictly configured to only accept incoming connections originating internally from the EC2 instance.

4.4 - Storage (Amazon S3)

- To optimize compute server load and reduce latency, Amazon Simple Storage Service (S3) is used as a content delivery mechanism.
- Static files (such as images, CSS, and JavaScript) are offloaded to an S3 bucket named cc-project-bucket.
- An IAM (Identity and Access Management) Role named cc_project_role_2026 with AmazonS3FullAccess policies was created and attached to the EC2 instance, securely allowing the web server to read and write media files directly to the bucket.

5.0 - Security & Cost Optimization

The infrastructure was engineered with strict adherence to the Principle of Least Privilege and

AWS Free Tier budget constraints.

Stateful Firewall Security (Security Groups):

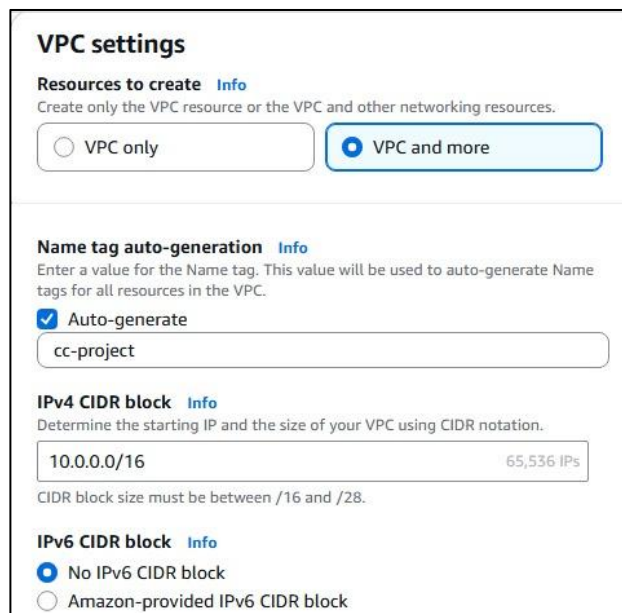
- Web Server Security Group (cc_project_web_sg): Configured to allow incoming HTTP traffic (Port 80) from anywhere (0.0.0.0/0) for public website visitors, and SSH traffic (Port 22) explicitly restricted to the administrator's local IP address for secure management.
- Database Security Group (cc_project_db_sg): Instead of whitelisting IP addresses, this group is configured to strictly accept MySQL traffic (Port 3306) where the source is exclusively the Web Server Security Group (cc_project_web_sg). This creates an impenetrable network lock where nothing from the public internet can reach the database directly.

Cost Management:

- All deployed compute and database resources utilized t3.micro instance types to remain within AWS Free Tier limitations.
- While high availability was maintained across two Availability Zones, the architecture was optimized for cost by deploying a NAT Gateway "In 1 AZ" (Single AZ), which significantly reduces hourly management charges for a student project while still servicing the private subnets.

6.0 - Step by Step Implementation Guide

6.1 - Step 1: VPC & Subnets



VPC settings

Resources to create [Info](#)
Create only the VPC resource or the VPC and other networking resources.

☐ VPC only ☒ VPC and more

Name tag auto-generation [Info](#)
Enter a value for the Name tag. This value will be used to auto-generate Name tags for all resources in the VPC.

☒ Auto-generate
cc-project

IPv4 CIDR block [Info](#)
Determine the starting IP and the size of your VPC using CIDR notation.

10.0.0.0/16 65,536 IPs
CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)
☒ No IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block

Number of Availability Zones (AZs) [Info](#)

Choose the number of AZs in which to provision subnets. We recommend at least two AZs for high availability.

1 | **2** | 3

► **Customize AZs**

Number of public subnets [Info](#)

The number of public subnets to add to your VPC. Use public subnets for web applications that need to be publicly accessible over the internet.

0 | **2**

Number of private subnets [Info](#)

The number of private subnets to add to your VPC. Use private subnets to secure backend resources that don't need public access.

0 | **2** | 4

► **Customize subnets CIDR blocks**

NAT gateways (\$) - *updated* [Info](#)

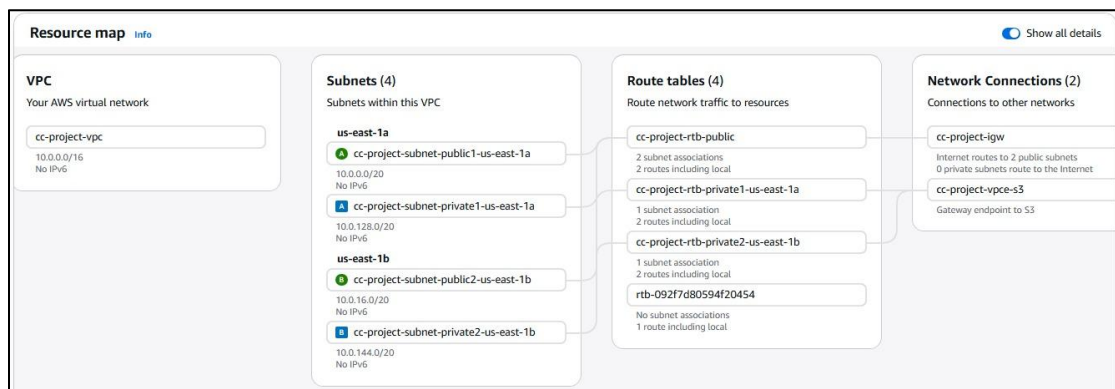
NAT gateway allows private resources to access the internet from any availability zone within a VPC, providing a single managed internet exit point for the entire region. Additional charges apply.

None | **Regional - new** | Zonal

VPC endpoints [Info](#)

Endpoints can help reduce NAT gateway charges and improve security by accessing S3 directly from the VPC. By default, full access policy is used. You can customize this policy at any time.

None | **S3 Gateway**

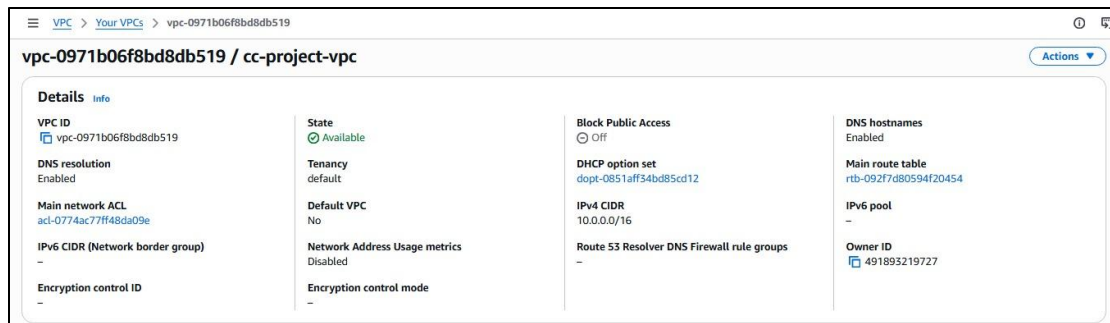


Create VPC workflow

>Create subnet 38%

▼ **Details**

- ✔ Create VPC: vpc-0971b06f8db519 [Link](#)
- ✔ Enable DNS hostnames
- ✔ Enable DNS resolution
- ✔ Verifying VPC creation: vpc-0971b06f8db519 [Link](#)
- ✔ Create S3 endpoint: vpce-05949623190fb0cc0 [Link](#)
- ✔ Create subnet: subnet-01d9f9e9504c8eef [Link](#)
- ✔ Create subnet: subnet-0a05d60159d524110 [Link](#)
- ✔ Create subnet: subnet-0328cca32c73d5d2a [Link](#)
- ✔ Create subnet
- ✔ Create internet gateway
- ✔ Attach internet gateway to the VPC
- ✔ Create route table
- ✔ Create route
- ✔ Associate route table
- ✔ Associate route table
- ✔ Create route table
- ✔ Associate route table
- ✔ Create route table
- ✔ Associate route table
- ✔ Verifying route table creation
- ✔ Associate S3 endpoint with private subnet route tables: vpce-05949623190fb0cc0 [Link](#)



Action Plan:

1. Navigate to VPC > Your VPCs.
2. Click Create VPC.
3. Select VPC and more (This creates subnets, route tables, and gateways automatically).
4. Name tag auto-generation: cc_group_project
5. IPv4 CIDR block: 10.0.0.0/16
6. Number of Availability Zones (AZs): Select 2 (for high availability).
7. Number of public subnets: 2
8. Number of private subnets: 2
9. NAT gateways: Select In 1 AZ (This is cheaper for a student project).
10. Click Create VPC.
11. Checkpoint: You should now see 1 VPC, 2 Public Subnets, 2 Private Subnets, 1 Internet Gateway (for public access), and 1 NAT Gateway (for private internet access).

6.2 - Step 2: Security Groups

Create the Web Server Security Group

Create security group info

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name info

Name cannot be edited after creation.

Description info

VPC info

Inbound rules info

Type	Protocol	Port range	Source	Description - optional
HTTP	TCP	80	Anyw... 0.0.0.0/0	Allow web traffic from internet Allow web traffic from internet
SSH	TCP	22	My IP 115.135.28.25/32	Allow SSH access only from my computer Allow SSH access only from my computer

[Add rule](#)

Outbound rules info

Type	Protocol	Port range	Destination	Description - optional
All traffic	All	All	Custom 0.0.0.0/0	

[Add rule](#)

sg-03278f559f974ae89 - cc_project_web_sg Actions

Details
Security group name
cc_project_web_sg
Owner
491893219727

Security group ID
sg-03278f559f974ae89
Inbound rules count
2 Permission entries

Description
Security group for web server - allows HTTP and SSH
Outbound rules count
1 Permission entry

VPC ID
vpc-0971b06f8bd8db519

[Inbound rules](#) [Outbound rules](#) [Sharing](#) [VPC associations](#) [Related resources - new](#) [Tags](#)

Inbound rules (2) Manage tags Edit inbound rules

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	
☐	-	sgr-048851b7ff3f313b0	IPv4	HTTP	TCP	80	0.0.0.0/0	AI
☐	-	sgr-0a085ccb36d02aa2b	IPv4	SSH	TCP	22	115.135.28.25/32	AI

Action Plan:

1. Security group name: cc_project-web-sg
2. Description: Security group for web server - allows HTTP and SSH
3. VPC: Select your cc_project VPC
4. Add Inbound Rules (This controls incoming traffic):
 - Rule 1 - HTTP (for website visitors): Type: HTTP, Port range: 80, Source: Anywhere-IPv4 (0.0.0.0/0).
 - Rule 2 - SSH (for you to manage the server): Type: SSH, Port range: 22, Source: My IP (This automatically adds your current IP address).
5. Outbound Rules: Leave as default (allows all outbound traffic).

Create the Database Security Group

Create security group Info

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details
Security group name Info
cc_project_db_sg
Name cannot be edited after creation.
Description Info
Security group for database - only accepts traffic from web server
VPC Info
vpc-0971b06f8bd8db519 (cc-project-vpc)

Inbound rules Info

Type Info Protocol Info Port range Info Source Info Description - optional Info

MySQL/Aurora TCP 3306 Custom sg-03278f559f974ae8 X Allow MySQL traffic only from web servers Delete

sg-03278f559f974ae8 X Allow MySQL traffic only from web servers

[Add rule](#)

Outbound rules Info

Type Info Protocol Info Port range Info Destination Info Description - optional Info

All traffic All All Custom 0.0.0.0/0 X

[Add rule](#)

sg-0bc6d43625b82b056 - cc_project_db_sg Actions

Details
Security group name
cc_project_db_sg
Owner
491893219727

Security group ID
sg-0bc6d43625b82b056
Inbound rules count
1 Permission entry

Description
Security group for database - only accepts traffic from web server
Outbound rules count
1 Permission entry

VPC ID
vpc-0971b06f8bd8db519

Inbound rules | Outbound rules | Sharing | VPC associations | Related resources - new | Tags

Inbound rules (1) Manage tags Edit inbound rules

☐	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Desc
☐	-	sgr-0e64693056a7ad1f5	-	MySQL/Aurora	TCP	3306	sg-03278f559f974ae89...	Allow

Action Plan:

1. Security group name: cc_project-db-sg
 2. Description: Security group for database - only accepts traffic from web server
 3. VPC: Same cc_project VPC
 4. Add Inbound Rule (Only ONE rule needed):
- Type: MySQL/Aurora
 - Port range: 3306
 - Source: Select "Custom", then select cc_group_project-web-sg.
 - Security Note: This means only resources using the cc_group_project-web-sg can access your database. Nothing from the internet can reach it directly.

6.3 - Step 3: Database Instance Creation

Create a Database Subnet Group

Create DB subnet group

To create a new subnet group, give it a name and a description, and choose an existing VPC. You will then be able to add subnets related to that VPC.

Subnet group details

Name

You won't be able to modify the name after your subnet group has been created.

cc_project-db-subnet-group

Must contain from 1 to 255 characters. Alphanumeric characters, spaces, hyphens, underscores, and periods are allowed.

Description

Private subnets for cc_group_project database

VPC

Choose a VPC identifier that corresponds to the subnets you want to use for your DB subnet group. You won't be able to choose a different VPC identifier after your subnet group has been created.

cc-project-vpc (vpc-0971b06f8bd8db519)

4 Subnets, 2 Availability Zones

Availability Zones
Choose the Availability Zones that include the subnets you want to add.

Choose an availability zone

us-east-1a X us-east-1b X

Subnets
Choose the subnets that you want to add. The list includes the subnets in the selected Availability Zones.

Select subnets

cc-project-subnet-private1-us-east-1a X Subnet ID: subnet-0328cca32c73d3d2a CIDR: 10.0.128.0/20

cc-project-subnet-private2-us-east-1b X Subnet ID: subnet-0188fecde31ed3e75 CIDR: 10.0.144.0/20

For Multi-AZ DB clusters, you must select 3 subnets in 3 different Availability Zones.

Subnets selected (2)

Availability zone	Subnet name	Subnet ID	CIDR block
us-east-1a	cc-project-subnet-private1-us-east-1a	subnet-0328cca32c73d3d2a	10.0.128.0/20
us-east-1b	cc-project-subnet-private2-us-east-1b	subnet-0188fecde31ed3e75	10.0.144.0/20

Action Plan:

1. Name: cc_project-db-subnet-group
2. Description: Private subnets for cc_group_project database
3. VPC: Select cc_project VPC
4. Add Subnets: Under Availability Zones, select 2 zones (us-east-1a, us-east-1b). Under Subnets, ONLY select the PRIVATE subnets from your VPC.

Create the Database Instance

Create database info

Free plan has access to limited features and resources
The free plan limits the features and resources that are available for RDS and Aurora databases. Upgrade your account plan to remove all limitations. [Learn more](#)

[Upgrade plan](#)

Engine options

Engine type info

☐ Aurora (MySQL Compatible) ☐ Aurora (PostgreSQL Compatible) ☒ MySQL ☐ PostgreSQL

☐ MariaDB ☐ Oracle ☐ Microsoft SQL Server ☐ IBM Db2

Choose a database creation method

☒ Full configuration
You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☐ Easy create
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Templates
Choose a sample template to meet your use case.

☐ Production
Use defaults for high availability and fast, consistent performance.

☐ Dev/Test
This instance is intended for development use outside of a production environment.

☒ Free tier
Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS.

Settings

Edition
☒ MySQL Community

Engine version [Info](#)
 View the engine versions that support the following database features.

► **Show filters**

Engine version
 MySQL 8.4.8

☐ **Enable RDS Extended Support** [Info](#)
 Amazon RDS Extended Support is a [paid offering](#). By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the [RDS for MySQL documentation](#).

DB instance identifier [Info](#)
 Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

cc-project-db

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Credentials Settings

Master username [Info](#)
 Type a login ID for the master user of your DB instance.

admin

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management
 You can use AWS Secrets Manager or manage your master user credentials.

☐ **Managed in AWS Secrets Manager - most secure**
 RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ **Self managed**
 Create your own password or have RDS create a password that you manage.

☐ **Auto generate password**
 Amazon RDS can generate a password for you, or you can specify your own password.

Master password [Info](#)

Password strength [Info](#) **Very strong**
 Minimum constraints: At least 8 printable ASCII characters. Can't contain any of the following symbols: / " * @

Confirm master password [Info](#)

▼ **Additional configurations**

Connectivity & security		
Endpoint & port	Networking	Security
Endpoint cc-project-db.cwb6gsswwzx6.us-east-1.rds.amazonaws.com	Availability Zone us-east-1b	VPC security groups cc_project_db_sg (sg-0bc6d43625b82b056) Active
Port 3306	VPC cc-project-vpc (vpc-0971b06f8bd8db519)	Publicly accessible Yes
	Subnet group cc_project-db-subnet-group	Certificate authority Info rds-ca-rsa2048-g1
	Subnets subnet-0328cca32c73d3d2a subnet-0188fecde31ed3e75	Certificate authority date May 26, 2061, 07:34 (UTC+08:00)
	Network type IPv4	DB instance certificate expiration date June 07, 2027, 05:18 (UTC+08:00)

Action Plan:

1. Select Standard Create.
2. Engine type: MySQL
3. DB instance identifier: cc-project-db
4. Master username: admin
5. Master password: cc_project_2026
6. DB Instance Class: db.t3.micro or db.t4g.micro (Free tier eligible)
7. Connectivity:

- VPC: cc_project VPC
- DB subnet group: cc_project-db-subnet-group

- Public access: Select NO (This keeps database off the internet)
 - VPC security group: Choose existing, SELECT cc_project-db-sg
8. Find Database Endpoint: Click on your database name (cc-group-project-db) and copy the endpoint: cc-project-db.cwb6gsswwzx6.us-east-1.rds.amazonaws.com.

6.4 - Step 4: Creating Your Web Server with EC2

Launch an instance Info

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags Info

Name

[Add additional tags](#)

▼ Application and OS Images (Amazon Machine Image) Info

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose **Browse more AMIs**.

Recents

Quick Start

Amazon Linux

aws

macOS

Mac

Ubuntu

Canonical Ubuntu

Windows

Microsoft

Red Hat

Red Hat

SUSE Linux

SUSE

Debian

debian

[Browse more AMIs](#)
 Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.1 AMI
 ami-00e801948462f718a (64-bit (x86), uefi-preferred) / ami-0b183bb1259186479 (64-bit (Arm), uefi)
 Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible

Description

Amazon Linux 2023 (kernel-6.1) is a modern, general purpose Linux-based OS that comes with 5 years of long term support. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Architecture	Boot mode	AMI ID	Publish Date	Username	Verified provider
64-bit (x86)	uefi-preferred	ami-00e801948462f718a	2026-05-21	ec2-user	

▼ Instance type Info | Get advice

Instance type

t3.micro
 Family: t3 2 vCPU 1 GiB Memory Current generation: true
 On-Demand Linux base pricing: 0.0104 USD per Hour
 On-Demand Windows base pricing: 0.0196 USD per Hour
 On-Demand Ubuntu Pro base pricing: 0.0139 USD per Hour
 On-Demand SUSE base pricing: 0.0104 USD per Hour On-Demand RHEL base pricing: 0.0392 USD per Hour

Free tier eligible

☐ All generations
[Compare instance types](#)

Additional costs apply for AMIs with pre-installed software

Create key pair

×

Key pair name

Key pairs allow you to connect to your instance securely.

cc_project-key

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ RSA
RSA encrypted private and public key pair

☐ ED25519
ED25519 encrypted private and public key pair

Private key file format

☒ .pem
For use with OpenSSH
 ☐ .ppk
For use with PuTTY

⚠ When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. [Learn more](#)

Cancel

Create key pair

▼ Network settings

Info

VPC - required

Info

vpc-0971b06f8bd8db519 (cc-project-vpc)

10.0.0.0/16

↻

Subnet

Info

subnet-01dfdfbe9504c8eef

cc-project-subnet-public1-us-east-1a

↻ Create new subnet

VPC: vpc-0971b06f8bd8db519

Owner: 491893219727

Availability Zone: us-east-1a (use1-az1)

Zone type: Availability Zone

IP addresses available: 4091

CIDR: 10.0.0.0/20

Auto-assign public IP

Info

Enable

Firewall (security groups)

Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☐ Create security group
 ☒ Select existing security group

Common security groups

Info

Select security groups

↻ Compare security group rules

cc_project_web_sg sg-03278f559f974ae89

×

VPC: vpc-0971b06f8bd8db519

Security groups that you add or remove here will be added to or removed from all your network interfaces.

User data - optional

Info

Upload a file with your user data or enter it in the field.

↑ Choose file

```
#!/bin/bash
# Update system
dnf update -y

# Install web server, PHP, and database tools
dnf install -y httpd php php-mysql mariadb105 wget unzip

# Start web server
systemctl start httpd
systemctl enable httpd

# Download sample PHP application
cd /var/www/html
wget https://github.com/vijaythapa333/php-crud-api/archive/refs/heads/main.zip
unzip main.zip
```

☐ User data has already been base64 encoded

Action Plan:

1. Name: cc_project-web-server
2. AMI: Select Amazon Linux 2023 AMI (Free tier eligible)
3. Instance Type: Select t3.micro
4. Key Pair (Login): Click Create new key pair, name it cc_group_project-key (RSA, .pem format), and download it.
5. Network Settings:

- VPC: cc_project VPC
- Subnet: Choose ANY Public Subnet
- Auto-assign public IP: Enable
- Firewall (security groups): Select existing security group cc_project-web-sg

6. Advanced Details (User Data): Paste the following script to automate installation:

```
#!/bin/bash
# Update system
dnf update -y

# Install web server, PHP, and database tools
dnf install -y httpd php php-mysqli mariadb105 wget unzip

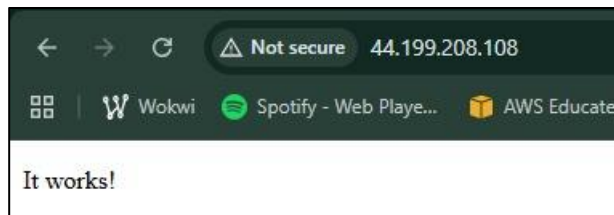
# Start web server
systemctl start httpd
systemctl enable httpd

# Download sample PHP application
cd /var/www/html
wget https://github.com/vijaythapa333/php-crud-api/archive/refs/heads/main.zip
unzip main.zip
cp -r php-crud-api-main/* .
rm -rf php-crud-api-main main.zip

# Set permissions
chown -R apache:apache /var/www/html
chmod -R 755 /var/www/html
```

6.5 - Step 5: Test Your Web Server (Before Database Connection)

Instances (1) Info							
Saved filter sets Choose filter set ▼		Find Instance by attribute or tag (case-sensitive)					
☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
☐	cc_project-web-server	i-0a06a7a3e0c5f3a6d	Running	t3.micro	Initializing	us-east-1a	ec2-44-199-208-108.co...



Action Plan:

1. Find your server's public IP in EC2 > Instances (44.199.208.108).
2. Open a web browser and paste the IP address.
3. Expected Result: A PHP error message about database connection (e.g., "Connection failed: php_network_getaddresses: getaddrinfo failed").
4. This means your web server is RUNNING but can't find the database yet. If you see ANY PHP output (even errors), your web server is working correctly!

```
PS C:\Users\Ameer Afiq\Downloads> ssh -i cc_project-key.pem ec2-user@44.199.208.108
The authenticity of host '44.199.208.108 (44.199.208.108)' can't be established.
ED25519 key fingerprint is SHA256:F3XL5hBRuZ9nTXsdKfBbVcZ6GdBd8QRxoPGnFvnlcQw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '44.199.208.108' (ED25519) to the list of known hosts.
```

```
#_
~\_ ##### Amazon Linux 2023
~~ \#####\
~~   \###|
~~     \|/_ https://aws.amazon.com/linux/amazon-linux-2023
~~~~ V~' '~>
      /
    _./_.
   _./_.
  _./m/'
```

```
[ec2-user@ip-10-0-9-153 ~]$
```

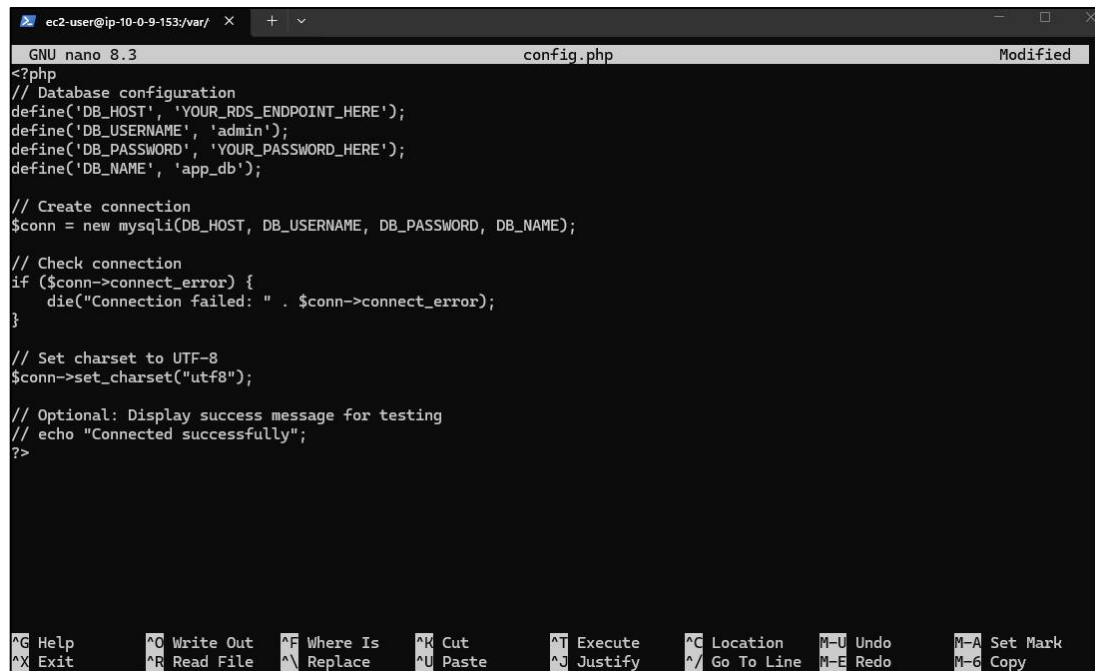
Action Plan:

Open command Prompt:

1. Navigate to the folder containing .pem key file
2. Run the ssh command:
`ssh -i "cc-project-key.pem" ec2-user@<Public-IP>`
3. Type yes when asked to confirm
4. Successful login will show the image above

Run the following command:

Edit database configuration file



```
GNU nano 8.3 config.php Modified
<?php
// Database configuration
define('DB_HOST', 'YOUR_RDS_ENDPOINT_HERE');
define('DB_USERNAME', 'admin');
define('DB_PASSWORD', 'YOUR_PASSWORD_HERE');
define('DB_NAME', 'app_db');

// Create connection
$conn = new mysqli(DB_HOST, DB_USERNAME, DB_PASSWORD, DB_NAME);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// Set charset to UTF-8
$conn->set_charset("utf8");

// Optional: Display success message for testing
// echo "Connected successfully";
?>
```

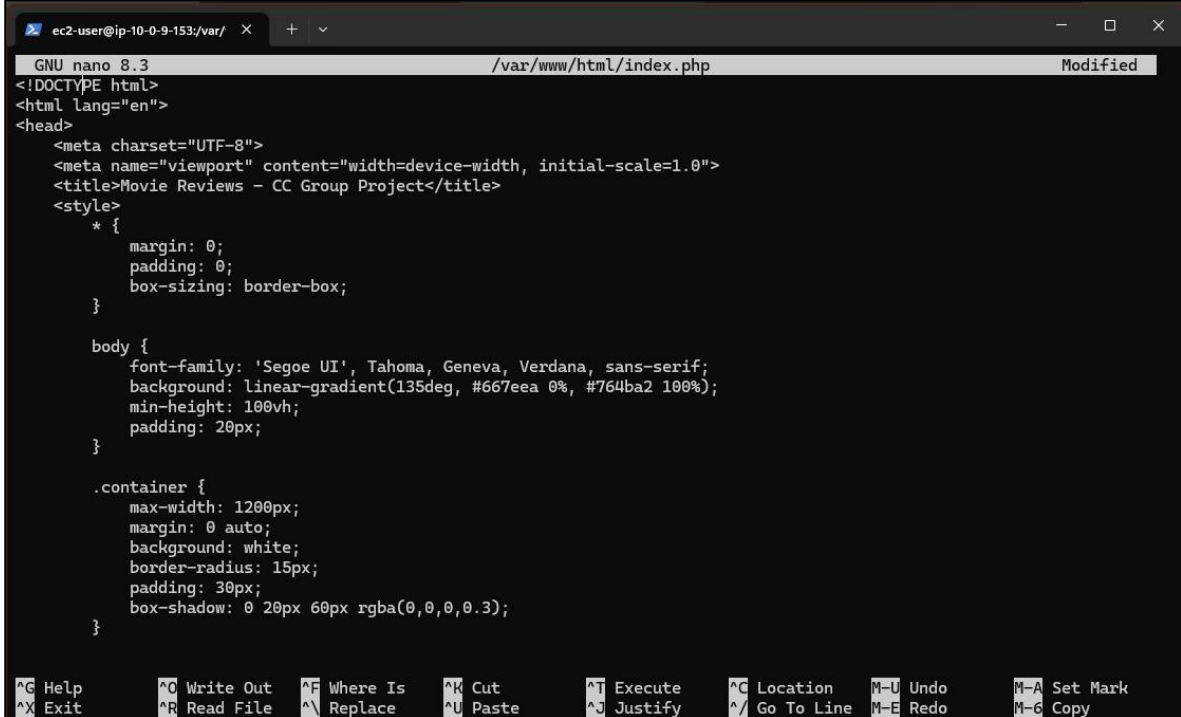
Set file permission

```
[ec2-user@ip-10-0-9-153 html]$ sudo chmod 644 config.php
[ec2-user@ip-10-0-9-153 html]$ sudo chown apache:apache config.php
```

Test database connection

```
[ec2-user@ip-10-0-9-153 html]$ php -r "include 'config.php'; if(isset(\$conn) && \$conn->ping()) echo 'Connected successfully to database'; else echo 'Connection failed';"
Connected successfully to database[ec2-user@ip-10-0-9-153 html]$ |
```

Edit main website file

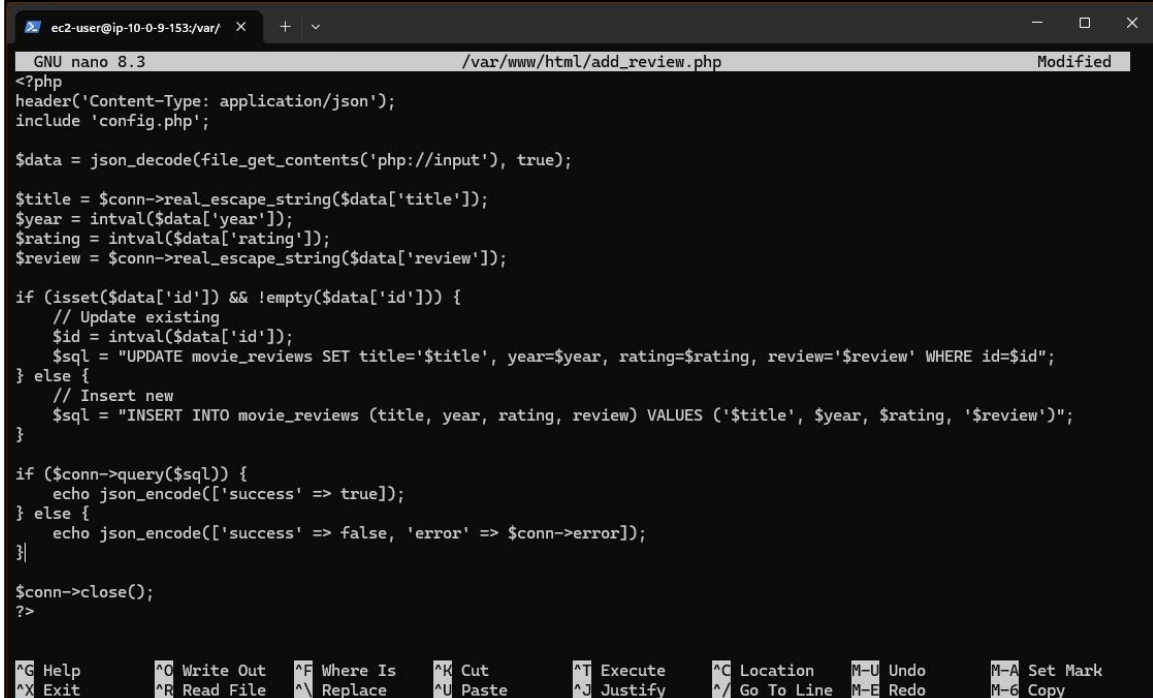


```
GNU nano 8.3 /var/www/html/index.php Modified
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Movie Reviews - CC Group Project</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      min-height: 100vh;
      padding: 20px;
    }

    .container {
      max-width: 1200px;
      margin: 0 auto;
      background: white;
      border-radius: 15px;
      padding: 30px;
      box-shadow: 0 20px 60px rgba(0,0,0,0.3);
    }
  </style>
</head>
<body>
  <div class="container">
    <h1>Movie Reviews</h1>
    <h2>CC Group Project</h2>
  </div>
</body>
</html>
```

Edit review processing file



```
GNU nano 8.3 /var/www/html/add_review.php Modified
<?php
header('Content-Type: application/json');
include 'config.php';

$data = json_decode(file_get_contents('php://input'), true);

$title = $conn->real_escape_string($data['title']);
$year = intval($data['year']);
$rating = intval($data['rating']);
$review = $conn->real_escape_string($data['review']);

if (isset($data['id']) && !empty($data['id'])) {
    // Update existing
    $id = intval($data['id']);
    $sql = "UPDATE movie_reviews SET title='$title', year=$year, rating=$rating, review='$review' WHERE id=$id";
} else {
    // Insert new
    $sql = "INSERT INTO movie_reviews (title, year, rating, review) VALUES ('$title', $year, $rating, '$review')";
}

if ($conn->query($sql)) {
    echo json_encode(['success' => true]);
} else {
    echo json_encode(['success' => false, 'error' => $conn->error]);
}

$conn->close();
?>
```

```
ec2-user@ip-10-0-9-153: /var/ + - x  
GNU nano 8.3 /var/www/html/delete_review.php Modified  
<?php  
header('Content-Type: application/json');  
include 'config.php';  
  
$data = json_decode(file_get_contents('php://input'), true);  
$id = intval($data['id']);  
  
$sql = "DELETE FROM movie_reviews WHERE id=$id";  
  
if ($conn->query($sql)) {  
    echo json_encode(['success' => true]);  
} else {  
    echo json_encode(['success' => false, 'error' => $conn->error]);  
}  
  
$conn->close();  
?>
```

Help **Write Out** **Where Is** **Cut** **Execute** **Location** **Undo** **Set Mark**
Exit **Read File** **Replace** **Paste** **Justify** **Go To Line** **Redo** **Copy**

Configure web application permission

```
[ec2-user@ip-10-0-9-153 html]$ sudo chown -R apache:apache /var/www/html  
[ec2-user@ip-10-0-9-153 html]$ sudo chmod -R 755 /var/www/html  
[ec2-user@ip-10-0-9-153 html]$ sudo systemctl restart httpd  
[ec2-user@ip-10-0-9-153 html]$
```

6.6 - Step 6: Create Amazon S3 Bucket

Create bucket [Info](#)
Buckets are containers for data stored in S3.

General configuration

AWS Region
US East (N. Virginia) us-east-1

Bucket type [Info](#)

☒ **General purpose**
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ **Directory**
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket namespace
Choose the namespace where you want to create your bucket. [Learn more](#)

☒ **Global namespace**
By default, S3 creates general purpose buckets in the global namespace.

☐ **Account Regional namespace (recommended)**
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name [Info](#)
cc-project-bucket

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#)

- Open the AWS Management Console and create Bucket.
- Enter a unique name and select AWS region

Object Ownership

info

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

Object Ownership

☒ ACLs disabled (recommended)

All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☐ ACLs enabled

Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

Object Ownership

Bucket owner enforced

Block Public Access settings for this bucket

Public access is granted to buckets and objects through access control lists (ACLs), bucket policies, access point policies, or all. In order to ensure that public access to this bucket and its objects is blocked, turn on Block all public access. These settings apply only to this bucket and its access points. AWS recommends that you turn on Block all public access, but before applying any of these settings, ensure that your applications will work correctly without public access. If you require some level of public access to this bucket or objects within, you can customize the individual settings below to suit your specific storage use cases. [Learn more](#)

☐ Block all public access

Turning this setting on is the same as turning on all four settings below. Each of the following settings are independent of one another.

☐ Block public access to buckets and objects granted through new access control lists (ACLs)
S3 will block public access permissions applied to newly added buckets or objects, and prevent the creation of new public access ACLs for existing buckets and objects. This setting doesn't change any existing permissions that allow public access to S3 resources using ACLs.

☐ Block public access to buckets and objects granted through any access control lists (ACLs)
S3 will ignore all ACLs that grant public access to buckets and objects.

☐ Block public access to buckets and objects granted through new public bucket or access point policies
S3 will block new bucket and access point policies that grant public access to buckets and objects. This setting doesn't change any existing policies that allow public access to S3 resources.

☐ Block public and cross-account access to buckets and objects through any public bucket or access point policies
S3 will remove all grants for buckets or access points with public access to buckets and objects across all accounts.

6.7 - Step 7: Create IAM roles for EC2 access to Amazon S3

- Navigate IAM roles and create roles
- Under Trusted Entity Type, select AWS Service
- Choose EC2 under use case and click next

Configure Roles Details

- Role name: cc_project_role_2026
- Description: Allows EC2 to read/write to S3 buckets for movie posters

Assign S3 Permissions

Step 2: Add permissions [Edit](#)

Permissions policy summary

Policy name	Type	Attached as
AmazonS3FullAccess	AWS managed	Permissions policy
AmazonS3ReadOnlyAccess	AWS managed	Permissions policy

- Search for AmazonS3FullAccess and select it.
- Click next

Verify role creation

Modify IAM role [Info](#)

Attach an IAM role to your instance.

Instance ID
i-0a06a7a3e0c5f3a6d (cc_project-web-server)

IAM role
Select an IAM role to attach to your instance or create a new role if you haven't created any. The role you select replaces any roles that are currently attached to your instance.

cc_project_role_2026 [Create new IAM role](#)

[Cancel](#) [Update IAM role](#)

- Return to IAM and search for cc_project_role_2026 and confirm that role has been created successfully and S3 policy is already attached

Website Interface output

Movie Review Hub
Share your thoughts about movies you've watched

[+ Add Movie Review](#)



67 Bomoh
Year: 2026
Rating: 6/10
67676767676767
[Delete](#)



ahahaha
Year: 2005
Rating: 5/10
no review
[Delete](#)

- This our application that we created called “Movie Review Hub”. This application allow user to browse movie information, write review and give rating to the movies.

Movie Review pop- up page

Add Movie Review

Movie Poster (Image):

Choose File

No file chosen

Upload JPG, PNG or GIF (max 5MB)

Submit Review

Cancel

A page where user can add and submit movie review. User also can choose a file to insert the movie image in JPG, PNG or GIF.

Final S3 Bucket Database

Amazon S3 > Buckets > cc-project-bucket-26 > movie-posters/

Amazon S3

<

▼ Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

▼ Files

File systems [New](#)

▼ Access management and security

Access Points

Access Points for FSx

Access Grants

IAM Access Analyzer

▼ Storage management and insights

Storage Lens

Batch Operations

Objects (8)

[Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

☐	Name	Type	Last modified	Size	Storage class
☐	1780826363_Screenshot2026-04-02131631.png	png	June 7, 2026, 17:59:24 (UTC+08:00)	778.4 KB	Standard
☐	1780826366_Screenshot2026-04-02131631.png	png	June 7, 2026, 17:59:28 (UTC+08:00)	778.4 KB	Standard
☐	1780826367_Screenshot2026-04-02131631.png	png	June 7, 2026, 17:59:30 (UTC+08:00)	778.4 KB	Standard
☐	1780827441_Screenshot2026-04-02131631.png	png	June 7, 2026, 18:17:22 (UTC+08:00)	778.4 KB	Standard
☐	1780827759_Screenshot2026-06-07011554.png	png	June 7, 2026, 18:22:40 (UTC+08:00)	97.8 KB	Standard
☐	1780828426_Screenshot2026-04-02131631.png	png	June 7, 2026, 18:33:48 (UTC+08:00)	778.4 KB	Standard
☐	1780828427_Screenshot2026-04-02131631.png	png	June 7, 2026, 18:33:48 (UTC+08:00)	778.4 KB	Standard
☐	1780828649_UniKL.jpg	jpg	June 7, 2026, 18:37:31 (UTC+08:00)	17.6 KB	Standard

7.0 - Testing & Validation

To ensure the architecture was functioning correctly, the team conducted a phased testing approach:

1. Initial Web Server Validation: After provisioning the EC2 instance, the public IPv4 address (44.199.208.108) was accessed via a web browser. The page successfully loaded but deliberately displayed a PHP connection error ("Connection failed: php_network_getaddresses"). This validated that the Apache service and PHP processor were running correctly prior to database binding.

2. Remote Administration Testing: The team successfully initiated an SSH connection into the EC2 instance using the generated cc-project-key.pem key file, verifying that the Security Group rules accurately permitted remote management from the administrator's IP.
3. Database Connectivity & Binding: Using the GNU Nano text editor via SSH, the config.php file was updated to include the generated RDS endpoint (cc-project-db.cwb6gsswwzx6.us-east-1.rds.amazonaws.com) and master credentials.
4. End-to-End Validation: A command-line PHP script was executed to ping the database, which returned "Connected successfully to database", definitively proving that the private subnet routing and database security groups were flawlessly configured. Finally, the web directory permissions were locked down (chmod 755 and chmod 644), and the httpd service was restarted to finalize the live environment.

8.0 - Challenges & Resolutions

During the deployment phase, the team encountered and systematically resolved the following technical roadblocks:

1. Challenge 1: Database Connection Timeouts
 - Issue: Initially, the EC2 instance could not communicate with the RDS database, and connection requests repeatedly timed out.
 - Resolution: We identified a misconfiguration in the Database Security Group, which was mistakenly set to accept traffic from our local workstation IP address instead of the internal Web Server. We resolved this by modifying the RDS Inbound Rules to explicitly reference the Custom EC2 Security Group ID, permanently fixing the internal VPC routing issue.
2. Challenge 2: S3 Object Access Denied
 - Issue: Static images (movie posters) hosted on the Amazon S3 bucket were failing to render on the live web application, returning "403 Forbidden" HTTP errors in the browser console.
 - Resolution: We discovered that simply unchecking the "Block Public Access" setting at the bucket level was insufficient to grant public viewing rights. The solution required writing and applying a custom JSON Bucket Policy that explicitly granted s3:GetObject permissions to wildcard (*) principals, allowing external users to view the assets without authenticating.

9.0 - Project Deliverables

- GitHub Repository: https://github.com/meerfiq/movie_review_hub_aws_cloud_computing_march26.git
- Repository contains: This detailed report, application source code (HTML/PHP), and database schema SQL files.
- Video Presentation: <https://youtu.be/gKsByHd794M>

- Video contains: A complete walk-through of our AWS console and architecture, presented collaboratively by all four team members.